

AI Assisted Coding

Lab Assignment - 9.4

Name: A. Sushmitha

Roll No: 2303A51541

Batch: 13

Lab 9 – Documentation Generation: Automatic Documentation and Code Comments

Task 1: Auto-Generating Function Documentation in a Shared Codebase

Scenario

You have joined a development team where several utility functions are already implemented, but the code lacks proper documentation. New team members are struggling to understand how these functions should be used.

Expected Outcome

- A Python script with well-structured Google-style docstrings
- Docstrings that clearly explain function behavior and usage
- Improved readability and usability of the codebase

Without Docstrings

One-Shot Code:

```
LAB-09 > 1.py > ...
1  def add(a, b):
2
3      return a + b
4  def subtract(a, b):
5
6      return a - b
7  def multiply(a, b):
8
9      return a * b
10 def divide(a, b):
11
12     if b == 0:
13         raise ValueError("Cannot divide by zero.")
14     return a / b
15 if __name__ == "__main__":
16     print(add(5, 3))      # Output: 8
17     print(subtract(5, 3)) # Output: 2
18     print(multiply(5, 3)) # Output: 15
19     print(divide(5, 3))   # Output: 1.6666666666666667
20     # Uncomment the below line to see the error handling
21     # print(divide(5, 0)) # Raises ValueError
22
```

With Docstrings:

```
LAB-09 > 1.py > divide
1  def add(a, b):
2      """
3      Adds two numbers and returns the result.
4      Args:
5          a (int or float): The first number.
6          b (int or float): The second number.
7      Returns:
8          int or float: The sum of a and b.
9      Example:
10         >>> add(5, 3)
11         8
12     """
13     return a + b
14 def subtract(a, b):
15     """
16     Subtracts the second number from the first number.
17     Args:
18         a (int or float): The number from which subtraction is performed.
19         b (int or float): The number to subtract.
20     Returns:
21         int or float: The result of a minus b.
22     Example:
23         >>> subtract(5, 3)
24         2
25     """
26     return a - b
27 def multiply(a, b):
28     """
29     Multiplies two numbers and returns the product.
30     Args:
31         a (int or float): The first number.
32         b (int or float): The second number.
33     Returns:
34         int or float: The product of a and b.
35     Example:
36         >>> multiply(5, 3)
37         15
```

```
38     """
39     return a * b
40 def divide(a, b):
41     """
42     Divides the first number by the second number.
43     Args:
44         a (int or float): The numerator.
45         b (int or float): The denominator.
46     Returns:
47         float: The result of division.
48     Raises:
49         ValueError: If b is zero.
50     Example:
51         >>> divide(5, 3)
52         1.6666666666666667
53     """
54     if b == 0:
55         raise ValueError("Cannot divide by zero.")
56     return a / b
57 if __name__ == "__main__":
58     print(add(5, 3))      # Output: 8
59     print(subtract(5, 3)) # Output: 2
60     print(multiply(5, 3)) # Output: 15
61     print(divide(5, 3))   # Output: 1.6666666666666667
62     # Uncomment the below line to see the error handling
63     # print(divide(5, 0)) # Raises ValueError
```

Output:

```
PS C:\AI Assisted Coding\LAB> & "C:\Program Files\Python311\python.exe" "c:/AI Assisted Coding/LAB/LAB-09/1.py"
8
2
15
2
15
1.6666666666666667
PS C:\AI Assisted Coding\LAB>
```

Zero-Shot Prompt:

Generate Google-style function docstrings, where each docstring should include: """ 1. A brief description of the functions 2. parameters with data types 3. return values 4. At least one example usage (if applicable). 5.

Experiment with different prompting styles, zero-shot or context-based, to observe quality differences. And call the functions at last.

Without Docstrings :

```
11.py > ...
1 #Generate a Python program that performs basic arithmetic operations (addition, subtraction, multiplication, division)
2 #on two numbers input by the user. The program should handle division by zero gracefully. zero shot.
3
4 def add(a, b):
5     return a + b
6
7 def subtract(a, b):
8     return a - b
9
10 def multiply(a, b):
11     return a * b
12
13 def divide(a, b):
14     if b == 0:
15         raise ValueError("Cannot divide by zero.")
16     return a / b
17
18 if __name__ == "__main__":
19     a = float(input("Enter first number: "))
20     b = float(input("Enter second number: "))
21
22     print("Addition:", add(a, b))
23     print("Subtraction:", subtract(a, b))
24     print("Multiplication:", multiply(a, b))
25
26     try:
27         print("Division:", divide(a, b))
28     except ValueError as e:
29         print("Error:", e)
30
```

With Docstrings :

```
3 def add(a, b):
4     return a + b
5     """Adds two numbers and returns the result.
6     Args:
7         a (int or float): The first number.
8         b (int or float): The second number.
9     Returns:
10        int or float: The sum of a and b."""
11 def subtract(a, b):
12     return a - b
13     """Subtracts the second number from the first number.
14     Args:
15         a (int or float): The number from which subtraction is performed.
16         b (int or float): The number to subtract.
17     Returns:
18        int or float: The result of a minus b."""
19 def multiply(a, b):
20     return a * b
21     """Multiplies two numbers and returns the product.
22     Args:
23         a (int or float): The first number.
24         b (int or float): The second number.
25     Returns:
26        int or float: The product of a and b."""
27 def divide(a, b):
28     if b == 0:
29         raise ValueError("Cannot divide by zero.")
30     return a / b
31     """Divides the first number by the second number.
32     Args:
33         a (int or float): The numerator.
34         b (int or float): The denominator.
35     Returns:
36        float: The result of division.
37     Raises:
38        ValueError: If b is zero."""
39 if __name__ == "__main__":
40     num1 = float(input("Enter the first number: "))
41     num2 = float(input("Enter the second number: "))
42     print(f"Addition: {add(num1, num2)}")
43     print(f"Subtraction: {subtract(num1, num2)}")
44     print(f"Multiplication: {multiply(num1, num2)}")
45     try:
46         print(f"Division: {divide(num1, num2)}")
47     except ValueError as e:
48         print(e)
```

Output:

```
PS C:\AI Assisted Coding\LAB> & "C:\Program Files\Python311\python.exe" "c:/AI Assisted Coding/LAB/11.py"
● Enter the first number: 9
Enter the second number: 7
Addition: 16.0
Subtraction: 2.0
Multiplication: 63.0
Division: 1.2857142857142858
○ PS C:\AI Assisted Coding\LAB>
```

Summary :

Adding proper Google-style docstrings significantly improves code readability and usability. They help new team members quickly understand the function purpose, inputs, outputs, and possible errors without analyzing the entire code logic. This makes the codebase more maintainable, professional, and suitable for collaborative development.

Task 2: Enhancing Readability Through AI-Generated Inline Comments

Scenario

A Python program contains complex logic that works correctly but is difficult to understand at first glance. Future maintainers may find it hard to debug or extend this code.

Expected Outcome

- A Python script with concise, meaningful inline comments
- Comments that explain why the logic exists, not what Python syntax does
- Noticeable improvement in code readability

Code With Inline Comments :

```
LAB-09 > 2.py > ...
1 def fibonacci(n):
2     if n <= 0:
3         return []
4     if n == 1:
5         return [0]
6     sequence = [0, 1]
7     for i in range(2, n):
8         # Each Fibonacci number is the sum of the previous two numbers
9         sequence.append(sequence[-1] + sequence[-2])
10    return sequence
11 def bubble_sort(arr):
12     n = len(arr)
13     for i in range(n):
14         swapped = False # Tracks whether any swaps happen in this pass
15         for j in range(0, n - i - 1):
16             # Compare adjacent elements and swap if out of order
17             if arr[j] > arr[j + 1]:
18                 arr[j], arr[j + 1] = arr[j + 1], arr[j]
19                 swapped = True
20         # If no swaps occurred, the list is already sorted (optimization)
21         if not swapped:
22             break
23     return arr
24 def binary_search(arr, target):
25     left = 0
26     right = len(arr) - 1
27     while left <= right:
28         mid = (left + right) // 2 # Middle index to divide search space
29         if arr[mid] == target:
30             return mid
31         # If target is greater, ignore left half
32         elif arr[mid] < target:
33             left = mid + 1
34         # If target is smaller, ignore right half
35         else:
36             right = mid - 1
37     return -1 # Return -1 if target not found
38 if __name__ == "__main__":
39     print("Fibonacci (first 10 numbers):")
40     print(fibonacci(10))
41     numbers = [64, 34, 25, 12, 22, 11, 90]
42     print("\nSorted List:")
43     sorted_numbers = bubble_sort(numbers)
44     print(sorted_numbers)
45     print("\nBinary Search:")
46     result = binary_search(sorted_numbers, 25)
47     print("Index of 25:", result)
```

Output :

```
PS C:\AI Assisted Coding\LAB> & "C:\Program Files\Python311\python.exe" "c:/AI Assisted Coding/LAB/LAB-09/2.py"
• Fibonacci (first 10 numbers):
[0, 1, 1, 2, 3, 5, 8, 13, 21, 34]

Sorted List:
[11, 12, 22, 25, 34, 64, 90]

Binary Search:
Index of 25: 3
PS C:\AI Assisted Coding\LAB> |
```

Table :

Requirement	How It Is Achieved
Insert inline comments	Added only inside logic sections
Avoid trivial comments	No comments for obvious syntax
Explain why, not what	Comments describe purpose & reasoning
Improve readability	Makes algorithms easier to understand
Avoid clutter	Minimal, meaningful comments only

Summery :

In this task, AI was used to improve the readability of a Python program containing loops, conditional logic, and algorithms such as Fibonacci, Bubble Sort, and Binary Search. The main goal was to insert inline comments only where the logic was complex or non-obvious.

Instead of explaining basic Python syntax, the comments focus on why the logic is written in a certain way, such as why a swap flag is used in Bubble Sort, why the search space is reduced in Binary Search, and how Fibonacci values are generated from previous elements.

This approach improves code clarity without cluttering the script with unnecessary explanations. As a result, the program becomes easier to understand, maintain, debug, and extend in a shared codebase.

Task 3: Generating Module-Level Documentation for a Python Package

Scenario

Your team is preparing a Python module to be shared internally (or uploaded to a repository). Anyone opening the file should immediately understand its purpose and structure.

Expected Outcome

- A well-written multi-line module-level docstring
- Clear overview of what the module does and how to use it
- Documentation suitable for real-world projects or repositories.

Code with Docstring :

```
1  #Generate a professional module-level docstring for the given Python file. The docstring should clearly describe the module's purpose, list any dependencies,
2  #summarize key functions or classes, and include a short example of how to use the module.
3  """
4  Module: calculator_module
5  This module provides basic arithmetic operations including addition,
6  subtraction, multiplication, and division. It is designed for simple
7  mathematical computations in educational, internal, or lightweight
8  applications.
9  Dependencies:
10     - Python 3.x
11     - No external libraries required
12  Functions:
13     add(a, b):
14         Returns the sum of two numbers.
15     subtract(a, b):
16         Returns the difference between two numbers.
17     multiply(a, b):
18         Returns the product of two numbers.
19     divide(a, b):
20         Returns the quotient of two numbers.
21         Raises a ValueError if division by zero is attempted.
22  Example Usage:
23     from calculator_module import add, subtract, multiply, divide
24     result = add(10, 5)
25     print(result) # Output: 15
26  """
27  def add(a, b):
28      """Return the sum of a and b."""
29      return a + b
30  def subtract(a, b):
31      """Return the difference of a and b."""
32      return a - b
33  def multiply(a, b):
34      """Return the product of a and b."""
35      return a * b
36  def divide(a, b):
37      """
38      Return the quotient of a and b.
39      Raises:
40          ValueError: If b is zero.
41      """
42      if b == 0:
43          raise ValueError("Cannot divide by zero.")
44      return a / b
```

```

45 if __name__ == "__main__":
46     # Example execution when run directly
47     print("Addition:", add(5, 3))
48     print("Subtraction:", subtract(5, 3))
49     print("Multiplication:", multiply(5, 3))
50     try:
51         print("Division:", divide(5, 3))
52         print("Division by Zero:", divide(5, 0))
53     except ValueError as e:
54         print("Error:", e)

```

Output :

```

PS C:\AI Assisted Coding\LAB> & "C:\Program Files\Python314\python.exe" "c:\AI Assisted Coding\LAB\LAB-09\3.py"
● Addition: 8
  Subtraction: 2
  Multiplication: 15
  Division: 1.6666666666666667
  Error: Cannot divide by zero.
○ PS C:\AI Assisted Coding\LAB>

```

Step 1: The module-level docstring is placed at the top of the file so anyone opening it immediately understands what the module does.

Step 2: It clearly explains the purpose (basic arithmetic operations) and lists dependencies (Python 3.x, no external libraries).

Step 3: It summarises key functions and provides a short usage example, making the module easy to understand, use, and maintain in a shared codebase.

Summary:

The improved module-level docstring provides a structured, professional overview that makes the module easier to understand, maintain, and reuse in a shared codebase. It enhances readability and ensures that developers can quickly grasp the module's purpose and functionality without reviewing the entire implementation.

Task 4: Converting Developer Comments into Structured Docstrings

Scenario

In a legacy project, developers have written long explanatory comments inside functions instead of proper docstrings. The team now wants to standardize documentation.

Expected Outcome

- A working Python script that processes another .py file
- Automatically inserted placeholder docstrings
- Clear demonstration of how AI can assist in documentation automation

Prompt:

Convert the explanatory inline comments into a structured Google-style docstring placed at the top of each function. Preserve the original meaning and intent of the comments.

Include:

A brief description, Args section with parameter types, Returns section with return type, Any important notes from the original comments, Remove redundant explanatory inline comments after conversion. Keep only necessary inline comments that clarify complex logic (if needed) and maintain professional formatting and consistency.

```
1  # This function calculates the nth Fibonacci number using iteration.
2  # Instead of recursion, we use a loop to avoid excessive memory usage.
3  # If n is less than or equal to 0, we return 0 since Fibonacci is undefined there.
4  # If n equals 1, we return 1.
5  # For all other cases, we build the sequence step by step.
6  def fibonacci(n):
7      if n <= 0:
8          return 0
9      elif n == 1:
10         return 1
11     prev, curr = 0, 1
12     for _ in range(2, n + 1):
13         temp = prev + curr
14         prev = curr
15         curr = temp
16     return curr
17 # This function performs binary search on a sorted list.
18 # It repeatedly divides the search interval in half.
19 # If the value of the search key is less than the item in the middle,
20 # it narrows the interval to the lower half.
21 # Otherwise, it narrows it to the upper half.
22 # If the target is found, return its index.
23 # If not found, return -1.
24 def binary_search(arr, target):
25     left = 0
26     right = len(arr) - 1
27     while left <= right:
28         mid = (left + right) // 2
29
30         if arr[mid] == target:
31             return mid
32         elif arr[mid] < target:
33             left = mid + 1
34         else:
35             right = mid - 1
36     return -1
37 # This function calculates total price including tax.
38 # It assumes each item in the list is a dictionary with a 'price' key.
39 # First, we calculate subtotal by summing all prices.
40 # Then we multiply subtotal by (1 + tax rate).
41 # Finally, we round to 2 decimal places.
```

```

42 def calculate_total_price(items, tax_rate=0.1):
43     subtotal = 0
44     for item in items:
45         subtotal += item['price']
46     total = subtotal * (1 + tax_rate)
47     return round(total, 2)
48 # This function validates email format.
49 # It checks whether '@' exists.
50 # Then it checks whether there is at least one '.' after '@'.
51 # This is a basic validation and does not follow full email RFC rules.
52 def validate_email(email):
53     if "@" not in email:
54         return False
55     domain_part = email.split("@")[-1]
56     if "." not in domain_part:
57         return False
58     return True
59 if __name__ == "__main__":
60     print("Fibonacci(7):", fibonacci(7))
61     numbers = [1, 3, 5, 7, 9, 11]
62     print("Binary Search (7):", binary_search(numbers, 7))
63     items = [{"price": 10.0}, {"price": 20.0}]
64     print("Total Price:", calculate_total_price(items))
65     print("Valid Email:", validate_email("user@example.com"))
66     print("Invalid Email:", validate_email("invalid.email"))
67

```

Summary:

The refactoring process does not change functional behavior, so program output remains the same. The improvement is structural and documentation-focused rather than execution-focused.

Task 5: Building a Mini Automatic Documentation Generator

Scenario

Your team wants a simple internal tool that helps developers start documenting new Python files quickly, without writing documentation from scratch.

Expected Outcome

- A working Python script that processes another .py file
- Automatically inserted placeholder docstrings
- Clear demonstration of how AI can assist in documentation automation

Prompt:

Generate a Python script that reads a .py file, detects all functions and classes, and automatically inserts placeholder Google-style docstrings for them if they don't already have one. The script should overwrite the file with the updated content. Keep it simple and use built-in libraries like ast.

```
LAB-09 > auto_doc_generator.py > ...
1 import ast
2 import sys
3 from pathlib import Path
4 class DocstringInserter(ast.NodeVisitor):
5     """Visitor to find functions and classes without docstrings."""
6     def __init__(self, source_lines):
7         self.source_lines = source_lines
8         self.insertions = [] # List of (line_number, docstring, indent)
9     def visit_FunctionDef(self, node):
10        self._check_and_add_docstring(node)
11        self.generic_visit(node)
12    def visit_AsyncFunctionDef(self, node):
13        self._check_and_add_docstring(node)
14        self.generic_visit(node)
15    def visit_ClassDef(self, node):
16        self._check_and_add_docstring(node, is_class=True)
17        self.generic_visit(node)
18    def _check_and_add_docstring(self, node, is_class=False):
19        """Check if node has docstring; if not, add placeholder."""
20        if ast.get_docstring(node) is not None:
21            return
22        # Get indentation
23        line_idx = node.lineno - 1
24        indent = len(self.source_lines[line_idx]) - len(
25            self.source_lines[line_idx].lstrip()
26        )
27        indent_str = " " * indent
28        # Generate placeholder docstring
29        if is_class:
30            docstring = self._create_class_docstring(node, indent_str)
31        else:
32            docstring = self._create_function_docstring(node, indent_str)
33        # Store insertion (append to list)
34        self.insertions.append((node.lineno, docstring, indent_str))
35    def _create_function_docstring(self, node, indent):
36        """Create placeholder docstring for function."""
37        args = [arg.arg for arg in node.args.args]
38        lines = [
39            f'{indent}    """Brief description of {node.name}."""',
40            f'{indent}',
41        ]
42        if args:
43            lines.append(f'{indent}    Args:')
44            for arg in args:
45                lines.append(f'{indent}        {arg}: Description of {arg}.')
46        lines.extend([
47            f'{indent}',
48            f'{indent}    Returns:',
49            f'{indent}        Description of return value.',
50            f'{indent}    ""',
51        ])
52    )
53    )
54    return "\n".join(lines)
```

```

55     def _create_class_docstring(self, node, indent):
56         """Create placeholder docstring for class."""
57         lines = [
58             f'{indent} """Brief description of {node.name}.'',
59             f'{indent}"""',
60             f'{indent} Attributes:',
61             f'{indent}     attribute_name: Description of attribute.',
62             f'{indent}"""',
63             f'{indent} Methods:',
64             f'{indent}     method_name: Description of method.',
65             f'{indent} """',
66         ]
67         return "\n".join(lines)
68     def process_file(filepath):
69         """Read, parse, and insert docstrings into Python file."""
70         path = Path(filepath)
71         if not path.exists():
72             print(f"Error: File '{filepath}' not found.")
73             sys.exit(1)
74         if not filepath.endswith(".py"):
75             print("Error: File must be a .py file.")
76             sys.exit(1)
77         # Read file
78         with open(path, "r", encoding="utf-8") as f:
79             content = f.read()
80             lines = content.splitlines(keepends=True)
81         # Parse AST
82         try:
83             tree = ast.parse(content)
84         except SyntaxError as e:
85             print(f"Error: Unable to parse file. {e}")
86             sys.exit(1)
87         # Find missing docstrings
88         visitor = DocstringInserter(content.splitlines())
89         visitor.visit(tree)
90         if not visitor.insertions:
91             print("No docstrings needed to be added.")
92             return
93         # Insert docstrings (reverse order to maintain line numbers)
94         for line_num, docstring, _ in sorted(visitor.insertions, reverse=True):
95             insertion_point = line_num
96             lines.insert(insertion_point, docstring + "\n")
97         # Write back to file
98         with open(path, "w", encoding="utf-8") as f:
99             f.writelines(lines)
100         count = len(visitor.insertions)
101         print(f"Success! Added {count} docstring(s) to '{filepath}'.")
102     if __name__ == "__main__":
103         if len(sys.argv) != 2:
104             print("Usage: python auto_doc_generator.py <filepath.py>")
105             sys.exit(1)
106         process_file(sys.argv[1])

```

```

PS C:\AI Assisted Coding\LAB\LAB-09> python auto_doc_generator.py 1.py
>>
No docstrings needed to be added.
PS C:\AI Assisted Coding\LAB\LAB-09>

```

Summary:

These tasks demonstrated how AI can assist in improving code readability and documentation by generating inline comments, structured docstrings, and documentation scaffolding automatically. It reduces manual effort and increases consistency across projects. However, human review is still essential to ensure accuracy and maintain quality standards.